# Task 12 – Re-evaluate and Adapt the Design (Day-4 Design Challenge)

Python Programming • Assignment 1 • Village Milk Co-operative Payment Calculator | Marks: 8 | Bloom's level: K5 – Evaluate

Name: **Dhaarani D** Register No.: _____ GitHub: _____ (folder Task12)

**Decision: MODIFY.** The module/function structure, Decimal money and the tests are retained; the step rule, the constants (a rate card per season) and the ten-day statement (a new bonus function) are changed. A full redesign is not needed.

## A. Impact Analysis (2 marks)

| What has changed | Affected part of the original design |
|---|---|
| Analyser sends fat and SNF with two decimals (e.g. 4.24) | **Expression** for steps: round((fat − 3.5)/0.1) now throws away up to half a step (Rs 0.20/L). **Rounding**: tie readings such as 3.55 occur often. Input validation already accepts decimals as text. |
| Seasonal rates: April–June base Rs 32.00 and fat step Rs 0.45 | **Constants** BASE_RATE and FAT_STEP_VALUE can no longer be single values; **functions** rate_per_litre and payout need to know the season. SNF step unchanged at Rs 0.30. |
| Bonus Rs 0.50 per litre if more than 500 litres in ten days | A **new function** and a **new constant pair**; the bonus depends on the ten-day litres, so it belongs in the ten-day statement, not in payout() for one entry. |

### How two-decimal readings change step calculation and rounding

A reading of 4.24 is 7.4 steps above 3.5. The original rule rounds this to 7 steps; with two-decimal readings the farmer's extra 0.04 % of fat is ignored, while 4.26 would be rounded up to 8 and overpay. The fair rule is **pro rata**: use the fractional step count (7.4) and round only the final rate or amount to the paisa. For fat 4.24 and SNF 8.58 (normal season) the original design gives Rs 33.10 per litre, the pro-rata rule Rs 33.20; on a 500 L delivery that is Rs 50.00 difference.

## B. Evaluation of the Original Design (2 marks)

| Part of original design | Still appropriate? | Worked example |
|---|---|---|
| Decimal money, ROUND_HALF_UP to paise, validation, tests | **Yes – retain** | No change needed: Decimal("4.24") parses exactly; payments are still whole paise. |
| Module with small functions | **Yes – retain** | Only the bodies of rate_per_litre and payout change; callers keep the same shape. |
| Single BASE_RATE / FAT_STEP_VALUE constants | **No – modify** | Summer 3.5/8.5 must be Rs 32.00, normal Rs 30.00: 10 L → 320.00 vs 300.00. |
| round() to a whole step | **No – modify** | fat 4.24 / SNF 8.58 → Rs 33.10 (old) vs Rs 33.20 (fair). Tie readings (3.55, 3.65, 3.75) give 0, 1, 2 steps with float round() (representation error and banker's rounding) but 1, 2, 3 with half-up; pro-rata steps avoid the problem. |
| payout() for one entry | **Partly** | Keep, but add season; bonus cannot be computed here because it needs all litres of the ten days. |

## C. Adaptation – revised design and working code (2 marks)

- **Rate cards in one dictionary** RATE_CARDS = {"normal": ..., "summer": ...}; a revision or a new season is a new entry.
- **season_for_month(month)** chooses the card (April = 4 to June = 6).
- **Season passed as a parameter** to rate_per_litre, payout and ten_day_statement (default "normal", so old calls still work).
- **Pro-rata steps** on Decimal: (fat − 3.5)/0.1 without rounding; money rounded once per payment.

- **Separate bonus(total_litres)** function: Rs 0.50 on all litres when the ten-day total exceeds 500 L (assumption: all litres, not only the excess; changing the rule affects one line).

*milkseason.py*

```
"""Season-aware payment calculator (changed requirement)."""
from decimal import Decimal as D, ROUND_HALF_UP

PAISA = D("0.01")
STD_FAT, STD_SNF, STEP = D("3.5"), D("8.5"), D("0.1")
SNF_STEP_VALUE = D("0.30")

RATE_CARDS = {                      # one entry per season: the ONLY place rates live
    "normal":   {"base": D("30.00"), "fat_step": D("0.40")},
    "summer":   {"base": D("32.00"), "fat_step": D("0.45")},    # April - June
}
BONUS_LITRES = D("500")             # more than this in ten days earns the bonus
BONUS_PER_LITRE = D("0.50")


def money(x):
    """Round to whole paise, half up."""
    return x.quantize(PAISA, rounding=ROUND_HALF_UP)


def season_for_month(month):
    """April (4) to June (6) is the summer season."""
    return "summer" if 4 <= month <= 6 else "normal"


def rate_per_litre(fat, snf, season="normal"):
    """Rate per litre; readings may have two decimals, so steps are fractional (pro rata)."""
    card = RATE_CARDS[season]
    fat_steps = (fat - STD_FAT) / STEP         # e.g. 4.23 -> 7.3 steps, not rounded to 7
    snf_steps = (snf - STD_SNF) / STEP
    return card["base"] + fat_steps * card["fat_step"] + snf_steps * SNF_STEP_VALUE


def payout(litres, fat, snf, season="normal"):
    """Amount for one delivery, before bonus, rounded to paise."""
    return money(litres * rate_per_litre(fat, snf, season))


def bonus(total_litres):
    """Bonus for ten days: Rs 0.50 on ALL litres if more than 500 litres were supplied."""
    return money(total_litres * BONUS_PER_LITRE) if total_litres > BONUS_LITRES else D("0.00")


def ten_day_statement(entries, season="normal"):
    """entries = list of (litres, fat, snf); returns (litres, milk amount, bonus, total)."""
    litres = sum((l for l, _, _ in entries), D("0"))
    milk = sum((payout(l, f, s, season) for l, f, s in entries), D("0.00"))
    b = bonus(litres)
    return litres, milk, b, milk + b
```

*run_season.py*

```
A. same reading, both seasons (10 L, fat 4.2, snf 8.7)
   normal  rate  33.40  pay   334.00
   summer  rate  35.75  pay   357.50
B. two-decimal analyser readings (10 L, fat 4.24, snf 8.58, normal season)
   old design (round to whole step): rate 33.10
   new design (pro rata steps)      : rate 33.20
   difference on a 500 L delivery   : Rs 50.00
C. bonus boundary (litres in ten days, all at 3.5/8.5, summer)
    499.9 L  milk   15996.80  bonus    0.00  total   15996.80
      500 L  milk   16000.00  bonus    0.00  total   16000.00
    500.1 L  milk   16003.20  bonus  250.05  total   16253.25
D. season from month
   month 3: normal
   month 4: summer
   month 6: summer
   month 7: normal
```

## D. Evidence-Based Defence (2 marks)

*test_season.py – tests for both seasons, two-decimal readings and the bonus*

```
Test                                Expected      Actual  Result
normal 10 L 3.5/8.5                   300.00      300.00  PASS
summer 10 L 3.5/8.5                   320.00      320.00  PASS
summer 10 L 4.2/8.7                   357.50      357.50  PASS
two-decimal 10 L 4.24/8.58 nrm        332.00      332.00  PASS
bonus at exactly 500 L                  0.00        0.00  PASS
bonus at 500.1 L                      250.05      250.05  PASS
bonus at 600 L                        300.00      300.00  PASS
statement 2 x 300 L summer          19500.00    19500.00  PASS
April is summer                       summer      summer  PASS
July is normal                        normal      normal  PASS
10 of 10 passed
```

| Evidence | What it shows |
|---|---|
| Both seasons (tests 1–3) | Normal 3.5/8.5 = 300.00 and summer = 320.00 for 10 L; summer 4.2/8.7 = Rs 35.75/L × 10 = 357.50 (= 32.00 + 7×0.45 + 2×0.30). |
| Two-decimal reading (test 4) | 4.24/8.58 gives Rs 33.20/L = 332.00 for 10 L, hand-calculated: 30 + 7.4×0.40 + 0.8×0.30. The old design would pay Rs 33.10. |
| Bonus (tests 5–8) | Exactly 500 L: no bonus (“more than 500”); 500.1 L: Rs 250.05; 600 L: Rs 300.00; two 300 L deliveries in summer: 19,200.00 + 300.00 = 19,500.00. |
| Season selection (tests 9–10) | April and June give summer, July gives normal. |

**Trade-offs.** *Gain:* the farmer is paid exactly for the quality measured, every season’s rates live in one table, the old tests carry over and 10 of 10 new tests pass. *Cost:* a more complex rate function and one more parameter everywhere; pro-rata steps make amounts depend on fractional steps, so the union must confirm that it wants pro-rata rather than rounding to 0.1 % steps. *Not chosen:* a full redesign (a class or database) would add code and risk with no gain for this change; keeping the original unchanged would underpay or overpay by up to Rs 0.35 per litre (0.20 from fat plus 0.15 from SNF) – Rs 175 on a 500 L delivery – and could not express the season or the bonus. *Assumptions to confirm:* the bonus applies to all litres, and the SNF step stays Rs 0.30 in every season.

## Evidence

Impact analysis, evaluation with worked examples, a working modified design and test results for both seasons and the bonus. Source: Task12/milkseason.py, run_season.py, test_season.py.